

SOFTWARE REQUIREMENTS SPECIFICATION

FOR

Secure File Sharing with Encryption

Prepared by

CHANDAN GOWDA C (241IT016)

PRABHAV P (241IT053)

SUCHETH K KATTE (241IT078)

**DEPARTMENT OF INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL**

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions, Acronyms and Abbreviations	3
1.4	References	4
1.5	Overview	4
2	Overall Description	4
2.1	Product Perspective	4
2.2	Product Functions	5
2.3	User Classes	6
2.4	Constraints	6
2.5	Assumptions and Dependencies	7
3	Specific Requirements	7
3.1	Functional Requirements	7
3.2	External Interface Requirements	11
3.2.1	User Interfaces	11
3.2.2	Hardware Interfaces	11
3.2.3	Software Interfaces	11
3.2.4	Communications Interfaces	11
3.3	Other Non-Functional Requirements	12
3.4	Constraints	13
	Appendix A	14
	A.1 Block Diagram	14
	Index	16
	Version History	17

1 Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe the functional and non-functional requirements of Secure File Sharing with Encryption, a web-based application that enables a user to upload, encrypt, queue, and deliver a file to a designated recipient through AES-256 file encryption whose key was established using Diffie-Hellman (DH) key exchange between the sender and the receiver. This SRS is intended for the development team, project reviewers, and evaluators as a reference for design, implementation, and testing of the system.

1.2 Scope

The described project implements a secure file sharing system, in which all files are encrypted with AES-256 on the sender's side prior to being uploaded on the server, thus ensuring that the server only stores encrypted data. Instead of a shared password, the communication uses a pair of private and public keys, with the Diffie-Hellman algorithm being utilized to allow each party to derive the shared secret key from their private key and the remote party's public key. If a receiver is not online at the time of sending, the encrypted blob is stored on the server until the receiver logs in, at which point the blob is transferred and purged from the server's storage. In this system, the server stores no sensitive information, only encrypted blobs (temporarily), public keys, and information related to transfers.

All client-server communication is carried over HTTPS, so every request and response is already protected in transit by Transport Layer Security (TLS): TLS encrypts each packet between the client and the server and prevents eavesdropping or tampering while the data is moving across the network. However, TLS protects data only *in transit*; once a packet reaches the server, TLS decrypts it, and if the intended recipient is not online, the file would otherwise sit on the server as plaintext until it is picked up. To close this gap, the system applies a second, independent layer of encryption at the application level: every file is encrypted end-to-end with an AES-256-GCM key derived from a Diffie-Hellman exchange between the sender and receiver *before* it is ever handed to the HTTPS/TLS channel for transport. As a result, the file remains encrypted with a key that only the sender and the intended receiver can reconstruct, both while it is in transit (protected additionally by TLS) and while it is sitting at rest in the server's delivery queue (protected solely by the DH-derived AES-256-GCM encryption, since TLS no longer applies once the connection terminates). This second, application-level encryption layer – and the fact that the server can queue and later deliver a file without ever being able to read it – is the central novelty of the project, going beyond the transport-level confidentiality that HTTPS alone would provide.

The current implementation covers the registration and login user flow, MFA, public key registration and utilization, blob encryption and upload, the Diffie-Hellman key exchange and the server-side blob storage and removal, as well as the admin panel and logging system. The delivery expiry date, delivery receipt, and key rotation features are to be implemented in the future.

1.3 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
AES-256	Advanced Encryption Standard, 256-bit key, used to encrypt the file contents.
AES-GCM	Galois/Counter Mode, authenticated encryption mode for AES, produces ciphertext and authentication tag, used to encrypt individual files with transfer key.
Diffie-Hellman (DH)	Public key cipher, allows two parties to agree on a shared secret over insecure channel, without exchanging any private information.
Public Key	One half of a DH key pair, registered with the server and shared with other users, used to establish shared secrets with this user.
Private Key	The other half of a DH key pair, stored on the device where the key was generated, never transmitted or stored on the server.
Ephemeral Key Pair	DH public/private key pair generated by the sender for a single file transfer, used with recipient's public key to establish transfer-specific shared secret.
Shared Secret	Secret value calculated independently by the sender (ephemeral private + recipient's public key) and the recipient (recipient's private + sender's ephemeral public key).
HKDF	HMAC-based Key Derivation Function, used to derive AES-256 transfer key from the raw DH shared secret.
Transfer Key (K)	AES-256 symmetric encryption key, derived from the DH shared secret, used for encrypting/decrypting a particular file.
Nonce / IV	Random value used in AES-GCM to ensure that ciphertexts are unique and secure, required for decrypting and verifying ciphertexts
Authentication Tag	Short value generated by AES-GCM to verify that the right transfer key was used, mismatch means wrong transfer key (and thus wrong DH shared secret).
Forward Secrecy	Property where compromise of one transfer's ciphertext, key or shared secret does not impact security of other transfers, achieved by using ephemeral DH key pairs for each transfer
Delivery Queue	Server-side temporary storage for an encrypted file blob, until the recipient is online and can receive it.
Metadata	Information about the file transfer, such as who sent it, to who, when, what transfer ID is, and other information, separate from the encrypted file blob
File Blob	Encrypted binary contents of a file currently waiting in the delivery queue on the server.
Transfer ID	Unique string assigned to each file transfer for record-keeping and auditing purposes.

Term / Acronym	Definition
REST API	Representational State Transfer Application Programming Interface, an architecture used for communication with the server.
HTTPS	Hypertext Transfer Protocol Secure, presents web pages securely, uses cryptography to prevent eavesdropping and tampering with data.
TLS	Transport Layer Security, the cryptographic protocol underlying HTTPS; secures data only while it is in transit between client and server, and does not protect data once it is stored on the server.

1.4 References

- [1] National Institute of Standards and Technology, “Advanced Encryption Standard (AES),” FIPS PUB 197, Nov. 2001.
- [2] W. Diffie and M. E. Hellman, “New Directions in Cryptography,” IEEE Transactions on Information Theory, vol. 22, no. 6, pp. 644–654, Nov. 1976.
- [3] National Institute of Standards and Technology, “Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography,” NIST SP 800-56A Rev. 3, Apr. 2018.
- [4] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” IETF RFC 5869, May 2010.
- [5] OWASP Foundation, “Cryptographic Storage Cheat Sheet.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html. [Accessed: Aug. 2, 2026].

1.5 Overview

The remainder of this document describes the system in progressively more detail. Section 2 provides an general description of the product, its features, target users and constraints. Section 3 details the functional and non-functional requirements, including interfaces, other non-functional requirements, and design constraints. Appendix A contains a block diagram of the encryption and queuing workflow based on Diffie-Hellmann approach, followed by an Index.

2 Overall Description

2.1 Product Perspective

Secure File Sharing with Encryption is a single purpose web application for exchanging files of confidential information between two registered users securely without requiring a shared password.

Each user registers by creating a long-term Diffie-Hellman (DH) key pair, storing their private key locally, and publishing their public key on the server. To send a file, a sender

initiates a session by generating a temporary ephemeral DH key pair, combining their private key and the recipient's public key, to produce a shared secret, which is then used to generate an Advanced Encryption Standard (AES)-256 key via the HMAC-based Key Derivation Function (HKDF). The file is then encrypted with AES-256-Galois/Counter Mode (GCM) and uploaded to a queue on the server. Once the message is delivered to the recipient, the file is removed from the queue. On receipt, the recipient decrypts the file using their private key and the sender's ephemeral public key to reproduce the shared secret, and subsequently the AES-256 key. Because AES-GCM authenticates as it decrypts, an incorrect key would result in an authentication tag check failure; there are no passwords, hashes, or verifiers involved. The server does not store any private keys, shared secrets, or plaintext files.

It is important to distinguish this application-level encryption from the transport-level security already provided by HTTPS. Every request between the client and the server travels over HTTPS, meaning the packet itself is encrypted in transit by TLS; this protects the data from network eavesdroppers while it moves between the sender, the server, and the receiver. TLS, however, terminates at the server: it says nothing about how data is protected once it is sitting on the server's disk. Since the receiver may not be online at the moment the sender uploads a file, the encrypted blob must be held in a delivery queue on the server for an indeterminate period, during which TLS offers no protection at all. This is precisely the gap that the DH-derived AES-256-GCM encryption is designed to close, and is the key novelty of the project: the file is encrypted end-to-end with a key that the server never possesses, so the queued blob remains confidential at rest on the server, not merely while in transit, and even the server operator cannot read its contents.

The product can be embedded into a larger document management and collaboration platform that requires secure file-sharing capabilities in an offline environment, in order to satisfy the requirement of transferring restricted or confidential information between users securely and privately, without a shared password, and without exposing the server to the plaintext file.

2.2 Product Functions

- **User Registration & Login:** registration involves creation of long-term DH key pair, and login involves establishing a secure session, potentially involving Multi-Factor Authentication (MFA).
- **Public-Key Directory:** the server stores long term public key for every user, which may be looked up by other users to send them encrypted messages.
- **File Upload & Encryption:** sender device generates ephemeral DH key pair and, combined with recipient's public key, derives AES-256 transfer key using HKDF, thereby encrypting the uploaded file.
- **Diffie-Hellman Secret Derivation:** sender uses ephemeral private key and recipient's public key to derive AES-256 transfer key; later recipient uses their private key and sender's ephemeral public key to derive the same transfer key.
- **Offline Delivery Queue:** The Server stores encrypted file blob for a recipient who is not online until they come back online.

- **Queued Delivery & Clearance:** Server delivers queued encrypted blob to recipient and subsequently deletes it from its storage.
- **Password-less Decryption:** Recipient decrypts file using transfer key derived from their private key and sender's ephemeral public key after verifying authenticity using AES-GCM authentication tag.
- **Metadata Logging:** Server logs sender, receiver, timestamp, transfer ID and status (pending/delivered), but not the decrypted contents of a transferred file.
- **Admin Visibility of Transfers:** Administrators can view logged metadata about file transfers.

2.3 User Classes

- **Senders:** Registered users who upload files addressed to a specific receiver, using their public key to encrypt those files.
- **Receivers:** Registered users who are able to receive queued encrypted files from senders, and decrypt those files with their private key.
- **Admins:** Users who can view metadata about individual file transfers for auditing purposes, but are not able to view any decrypted content or cryptographic keys.

2.4 Constraints

- **C.1 Regulatory Policies** – The system must comply with data-protection principles such as not retaining plaintext files, private keys or shared secrets.
- **C.2 Cryptographic Constraints** – Only Diffie-Hellman (or Elliptic-Curve Diffie-Hellman) key exchange for shared-secret establishment, HKDF for transfer-key derivation, and AES-256-GCM (authenticated encryption) for file encryption shall be used. No password, password hash or separate verifier shall ever be stored; the correctness of a decryption attempt is established solely through AES-GCM authentication-tag verification.
- **C.3 Software Interfaces** – Dependent on a cryptographic library (e.g., PyCryptodome/libsodium) for DH/ECDH, HKDF and AES-256-GCM implementations.
- **C.4 Communications** – All client-server communication must occur over HTTPS. This HTTPS/TLS layer secures each packet only while it is in transit between client and server; it does not extend to data held at rest on the server. Because a file may remain queued on the server for an indeterminate time while the receiver is offline, the application-level Diffie-Hellman-derived AES-256-GCM encryption described in C.2 must be applied independently of, and in addition to, HTTPS/TLS, so that the queued file blob stays confidential even though TLS protection ends at the server boundary.
- **C.5 Audit Functions** – The system must log every transfer (sender, recipient, timestamp, transfer ID, status) in the metadata log.

- **C.6 Storage Constraint** – The server shall store only queued encrypted file blobs (until delivered), sender/receiver ephemeral and long-term public keys, file nonces and authentication tags, and metadata; it shall never store private keys, plaintext files, derived shared secrets, or transfer keys. Once a queued blob is delivered to the receiver, the server shall delete its stored copy of that blob.
- **C.7 Security Considerations** – Login and multi-factor authentication are required functional components of the system; the DH key pair used for file encryption is independent of the account login password and is never derived from it.

2.5 Assumptions and Dependencies

- It is assumed that the sender has already established an authenticated session before initiating an upload.
- It is assumed that each user’s long-term public key, once registered, is authentic (i.e., the system assumes no man-in-the-middle substitution of a registered public key).
- The system depends on a cryptographically secure random number generator for generating ephemeral DH key pairs.
- The system depends on the availability of a reliable DH/ECDH implementation and a consistent HKDF configuration across upload and download.
- Internet connectivity is assumed for the sender at upload time and for the receiver at the time of coming online to retrieve the queued file.
- If the underlying cryptographic library changes, the requirements concerning the DH group/curve parameters and AES mode must be revisited.

3 Specific Requirements

3.1 Functional Requirements

F.1 User Registration

Description: The system shall allow a new user to register by providing a unique username, email address and account password, and shall generate a long-term DH public/private key pair on the user’s device at registration time, registering only the public key with the server. The account password shall be processed using a secure password-hashing function before being stored, and shall never be used to derive any file-encryption key.

Input: Username, email address, account password.

Output: Confirmation of successful registration, generated long-term DH key pair (private key retained client-side only), or an error message.

F.2 User Login

Description: The system shall authenticate registered users and then establish a session before uploading, checking for queued items, or downloading the files. The login process is a prerequisite operation and shall be performed before a user can perform any file operations.

Input: Username/email and account password.

Output: Authenticated session or login failure message.

F.3 Multi-Factor Authentication (MFA)

Description: The system shall require an additional authentication factor (e.g., one-time code) after a user's password is successfully validated to establish a login session. MFA is mandatory and applicable to all accounts.

Input: Account password (already validated) and a one-time verification code.

Output: Fully authenticated session or MFA failure message.

F.4 Public Key Directory Lookup

Description: The system shall allow an authenticated sender to perform a lookup by username or email to discover the long-term public key of an intended receiver which is stored in the key directory, to be used in the Diffie-Hellman exchange process.

Input: Receiver's username or email.

Output: Receiver's registered long-term public key, or a not-found message.

F.5 Upload File to a Receiver

Description: The system shall allow an authenticated user (the sender) to select a file and an intended receiver for secure, queued delivery. No password is requested from either party.

Input: File to be uploaded and the intended receiver's identity.

Output: Acknowledgement that the file has been received for encryption and queuing.

F.6 Generate Ephemeral DH Key Pair

Description: The system shall generate a fresh, cryptographically random ephemeral DH public/private key pair on the sender's device, uniquely for this transfer, to provide forward secrecy.

Input: Upload event trigger.

Output: Ephemeral DH key pair (private key held only in sender's memory for this transfer; public key to accompany the transfer).

F.7 Derive Shared Secret and Transfer Key (Sender Side)

Description: The system shall combine the sender’s transient private key, computed in F.6, with the receiver’s registered long-term public key, extracted in F.4, using a Diffie-Hellman key exchange to establish a shared secret, from which a 256-bit AES transfer key (K) shall be derived using HKDF.

Input: Sender’s ephemeral private key, receiver’s long-term public key.

Output: AES-256 transfer key (K), held only in memory on the sender’s device.

F.8 Encrypt File with AES-256-GCM

Description: The system shall encrypt the uploaded file using the transfer key (K) in GCM (authenticated) mode before any data is written to persistent storage, producing ciphertext, a nonce and an authentication tag.

Input: Plaintext file and transfer key (K).

Output: Encrypted file blob, file nonce and file authentication tag.

F.9 Queue Encrypted File for Delivery

Description: The system shall place the encrypted file blob, its nonce and authentication tag, and the sender’s ephemeral public key into a delivery queue on the server, associated with the intended receiver, and shall mark the corresponding metadata record’s status as pending. Because HTTPS/TLS no longer protects the data once the connection to the client has terminated, the blob stored in this queue remains confidential only by virtue of the DH-derived AES-256-GCM encryption already applied in F.7–F.8; the server is never able to decrypt it.

Input: Encrypted file blob, nonce/tag, sender’s ephemeral public key, receiver identity.

Output: Confirmation that the transfer has been queued, unique transfer ID.

F.10 Detect Receiver Online and Trigger Delivery

Description: The system shall detect when the intended receiver next establishes an authenticated session (comes online) and shall automatically trigger delivery of any file(s) queued for that receiver.

Input: Receiver login/session event.

Output: List of pending transfer(s) to be delivered.

F.11 Deliver Queued File and Clear Server Storage

Description: Upon delivery, the blob of the encrypted file, its nonce/AuthTag, and the ephemeral public key of the sender should be sent to the receiver’s device over HTTPS, and the file blob stored locally on the sender’s device should be deleted immediately after a successful delivery attempt, and the transfer’s metadata should be updated to reflect the delivery status.

Input: Queued encrypted file blob, nonce/tag, sender's ephemeral public key, delivery confirmation from receiver.

Output: Encrypted blob removed from server storage; metadata status updated to delivered.

F.12 Derive Shared Secret and Transfer Key (Receiver Side)

Description: On receiving a delivered file, the system combines the receiver's own long-term private key with the sender's ephemeral public key (from F.11), which was encrypted with the receiver's long-term public key, using Diffie-Hellman key exchange to produce the same shared secret as in F.7 and derives the same 256-bit AES transfer key (K) from it with HKDF. Neither passwords nor verifiers are stored; their correctness is established by the authentication-tag check in F.13.

Input: Receiver's long-term private key, sender's ephemeral public key.

Output: Recovered AES-256 transfer key (K), held only in memory on the receiver's device.

F.13 Decrypt and Download File

Description: Using the transfer key (K) extracted in F.12, the system shall decrypt the encrypted file blob using AES-256-GCM, verifying the file 's authentication tag before producing any plaintext output, and provide the decrypted file to the recipient for download. In the event that verification process results in a mismatch of the recomputed authentication tag, the file shall be rejected with no indication of whether the key material or the stored information was incorrect.

Input: Encrypted file blob, file nonce, file authentication tag, and recovered transfer key (K).

Output: Decrypted original file delivered to the receiver, or an authentication/integrity failure message.

F.14 Metadata Logging

Description: The system shall log every upload, queue, delivery and download event as a metadata record (sender, receiver, transfer ID, timestamp, status) without storing any plaintext file content, private key, or shared secret.

Input: Transfer event (upload, queued, delivered, or downloaded).

Output: Updated metadata log entry.

F.15 Admin Transfer Log Viewing

Description: The system shall allow an administrator to view the transfer log, including sender username, receiver username, unique transfer ID, timestamp and status (pending/delivered), without exposing plaintext files, private keys or shared secrets.

Input: Administrator request to view logs.

Output: List of transfer log entries.

F.16 Error Handling

Description: The system shall detect and report errors such as failed authentication-tag verification, corrupted uploads, an unregistered receiver public key, or storage/queue failures, with clear, non-revealing error messages that do not leak cryptographic material.

Input: Failed operation event.

Output: User-facing error message and internal error log entry.

3.2 External Interface Requirements

3.2.1 User Interfaces

Description: A web-based graphical user interface shall be provided for account registration/login (including automatic DH key-pair generation), multi-factor authentication, receiver lookup, file upload for queued delivery, and download of received files. The interface shall clearly indicate encryption status and pending/delivered status of each transfer.

Input: User credentials, MFA verification codes, files, receiver identities.

Output: Upload confirmations, pending/delivered status, download results, and error messages.

3.2.2 Hardware Interfaces

Description: The system shall run on standard server hardware capable of performing AES-256, Diffie-Hellman and HKDF computations, and shall be accessible from any client device with a modern web browser and internet connectivity.

3.2.3 Software Interfaces

Description: The system shall interface with a cryptographic library providing DH/ECDH, HKDF and AES-256 implementations, a database for metadata and public-key storage, and a temporary file/object storage service for queued encrypted blobs.

Input: Cryptographic library calls, database queries, storage read/write/delete requests.

Output: Encrypted data, stored metadata records, retrieved encrypted blobs, storage-deletion confirmations.

3.2.4 Communications Interfaces

Description: The system shall provide communication between the frontend and backend through RESTful APIs over HTTPS. All requests and responses shall use JSON for parameters, while encrypted file blobs and public keys shall be exchanged via secure upload/queue/download endpoints. This HTTPS/TLS channel provides transport-level

confidentiality and integrity for every request and response; it is distinct from, and does not substitute for, the application-level DH-derived AES-256-GCM encryption applied to file contents, which is what protects a queued blob while it is stored at rest on the server awaiting delivery.

Input: API requests including registration, login, MFA verification, public-key lookup, upload, queue status, and download.

Output: API responses containing confirmations, public keys, queue status, or encrypted file data.

3.3 Other Non-Functional Requirements

NF.1 Performance

Description: Support 100+ simultaneous sessions; encrypt/decrypt files up to 50 MB within 2 seconds; DH shared-secret and HKDF derivation within 1 second; 95% of API requests served within 1 second; deliver queued files within 5 seconds of the receiver coming online.

NF.2 Database

Description: Store user records (username, email, hashed account password, MFA status, long-term DH public key) and transfer metadata (transfer ID, sender, receiver, timestamp, status pending/delivered, sender's ephemeral public key, file nonce/tag). Never store private keys, plaintext files, shared secrets or transfer keys. Maintain referential integrity between users and transfers, and remove the encrypted blob reference once delivered.

NF.3 Reliability

Description: Guarantee correct decryption given the correct DH key pairs on both sides; detect and reject corrupted encrypted blobs on delivery or download; guarantee that a queued file is not lost before the receiver comes online.

NF.4 Availability

Description: Target 99.5% uptime for upload, queuing and delivery services, with periodic backups of metadata and of encrypted blobs that are still pending delivery.

NF.5 Security

Description: Encrypt every file with AES-256-GCM using a key derived from a Diffie-Hellman shared secret unique to each transfer; use a fresh ephemeral key pair per transfer for forward secrecy; require MFA after login; never persist plaintext files, private keys, shared secrets or transfer keys; delete the server's copy of an encrypted blob immediately after delivery; secure all traffic with HTTPS. HTTPS/TLS and the DH-derived AES-256-GCM encryption serve two distinct, complementary purposes: TLS secures packets

in transit between client and server, while the DH-derived AES-256-GCM layer secures the file itself, including while it sits at rest in the server’s delivery queue – a period during which TLS offers no protection. This two-layer defense-in-depth design, in which the server can hold a file it cannot itself decrypt, is the core security novelty of the system.

NF.6 Maintainability

Description: Isolate the cryptographic module (DH key-pair generation, shared-secret derivation, HKDF, AES) from storage, queuing and API layers to allow independent updates.

NF.7 Portability

Description: Deploy on standard Linux servers using widely available, well-maintained cryptographic libraries.

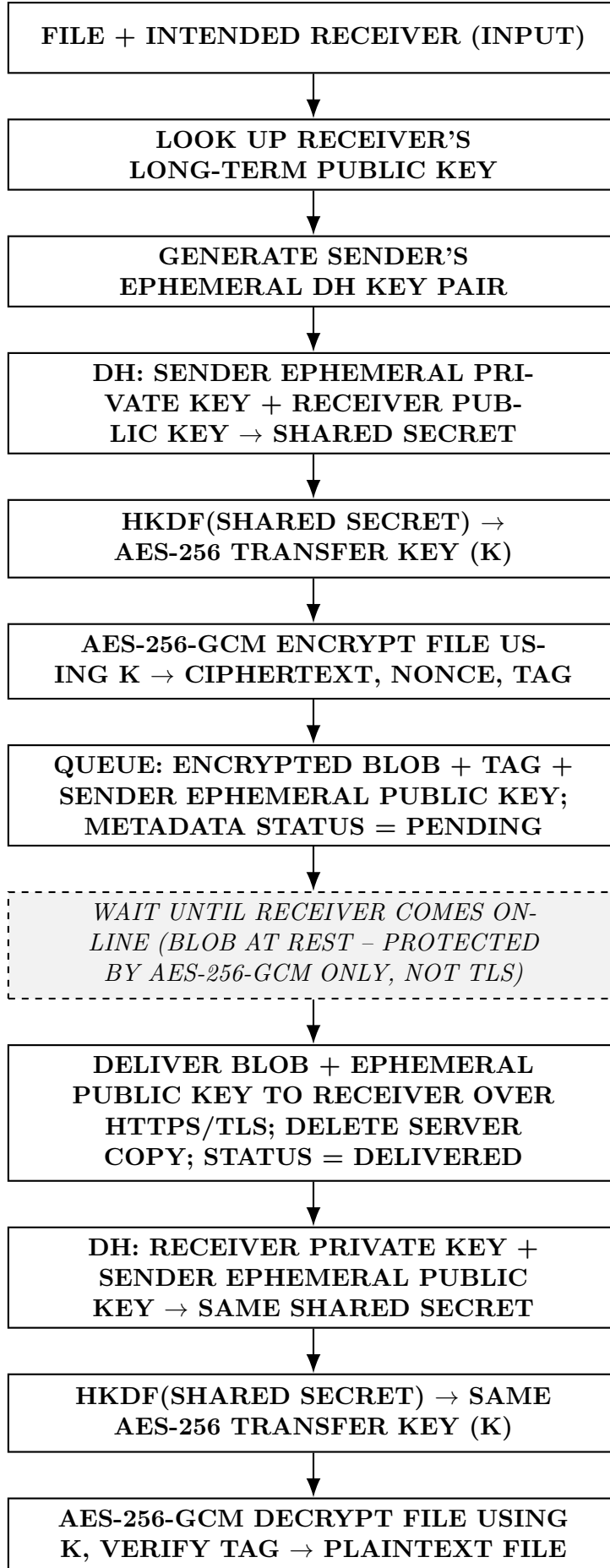
3.4 Constraints

- **Cryptographic Compliance:** AES-256 must be used in authenticated (AES-GCM) mode to encrypt each file, so that decryption failures caused by an incorrect transfer key are detected via authentication-tag mismatch rather than silently returning garbage plaintext.
- **Key-Exchange Compliance:** A Diffie-Hellman (or Elliptic-Curve Diffie-Hellman) exchange must be used for every transfer, combining the sender’s ephemeral private key with the receiver’s long-term public key on the sender’s side, and the receiver’s long-term private key with the sender’s ephemeral public key on the receiver’s side, to derive an identical shared secret on both ends.
- **Password-Free File Encryption:** The system shall not use or store any password, password hash or verifier for file encryption; correctness shall be determined only by whether AES-GCM authentication-tag verification succeeds on decryption.
- **Authentication Requirement:** Login and multi-factor authentication shall both be implemented as mandatory functional components of the system, independent of the DH key pair used for file encryption.
- **Storage Clearance:** The server shall delete its stored copy of an encrypted file blob immediately upon successful delivery to the receiver.
- **Transport Security:** All communication must occur over HTTPS/TLS. This transport-layer security is necessary but not sufficient on its own: it protects packets only while in transit, so it must be paired with the application-layer DH-derived AES-256-GCM file encryption to protect files that are queued at rest on the server.

Appendix A

A.1 Block Diagram

The diagram below describes the file transfer process from the sender's perspective. The sender selects the file to be transferred and specifies the recipient. They generate a one-time ephemeral key pair and use the public key of the recipient along with their own ephemeral private key to derive a shared secret via the Diffie-Hellman key exchange. Otherwise stated, HKDF (HMAC-based Extract-and-Expand Key Derivation) is applied to the shared secret to obtain an AES-256 key. Finally, the file is encrypted using AES-256-GCM and stored as a blob with the sender's ephemeral public key on the server with a pending status. Neither passwords nor any sort of verifier are stored. Note that this application-level encryption is applied on top of, and independently of, the HTTPS/TLS transport encryption used for every network exchange: TLS secures the packet while it travels between client and server, whereas the AES-256-GCM encryption shown below secures the file itself, including while it waits in the server's delivery queue for an offline recipient. When the file transfer process is initiated, the blob is delivered to the server, where it awaits for the recipient to come online. Once such a request is received, the server forwards the encrypted message to the recipient, deletes the blob, and updates the transfer status to delivered. To decrypt the file, the recipient applies their private key along with the sender's ephemeral public key to re-derive the shared secret and, subsequently, the AES key. The encryption algorithm's authentication tag is used to verify whether the derived key is correct, with an incorrect tag causing the decryption process to fail. Notably, the correct decryption key can only be obtained by combining the recipient's private key with the sender's ephemeral public key.



Index

AES-256, [3](#)
AES-GCM, [3](#)
Authentication Tag, [3](#)

Delivery Queue, [3](#)
Diffie-Hellman (DH), [3](#)

Ephemeral Key Pair, [3](#)

Forward Secrecy, [3](#)

HKDF, [3](#)

Metadata, [3](#)

Nonce / IV, [3](#)

Private Key, [3](#)
Public Key, [3](#)

Security, [12](#)
Shared Secret, [3](#)

TLS, [4](#)
Transfer Key (K), [3](#)

Version History

Version	Last Updated	Reason for Change
1.0	03-08-2026	Initial Update
1.1	09-08-2026	Clarified relationship between HTTPS/TLS transport-layer encryption and the application-layer DH-derived AES-256-GCM encryption; highlighted the latter as the project's core novelty for protecting queued files at rest.